DSCI 551 Course Final Report

Project: Data Pipeline Monitoring & Lineage Tracking System

Team Members: Yu-Huan Yu- yuhuanyu@usc.edu

---

1. Introduction & Motivation

During my time working as a data engineer, one challenge remained consistently unresolved: managing metadata for hundreds of interconnected ETL pipelines. When a pipeline failed, it was difficult to determine the root cause, assess the downstream impact, or understand which tables were affected. Existing tools either lacked the query expressiveness to traverse graph-like dependencies or could not handle the concurrent read-write workloads inherent to live monitoring dashboards.

This project uses **PostgreSQL** as its sole database engine. The project models a data engineering environment with hundreds of interconnected ETL pipelines. The database stores four categories of data: dataset catalog metadata, pipeline definitions, directed lineage edges between tables, and high-frequency pipeline execution logs. At the current scale, the schema holds 50 datasets, 100 pipelines, 200 lineage edges, and over 1 million execution records simulating one year of hourly job runs across the full pipeline fleet.

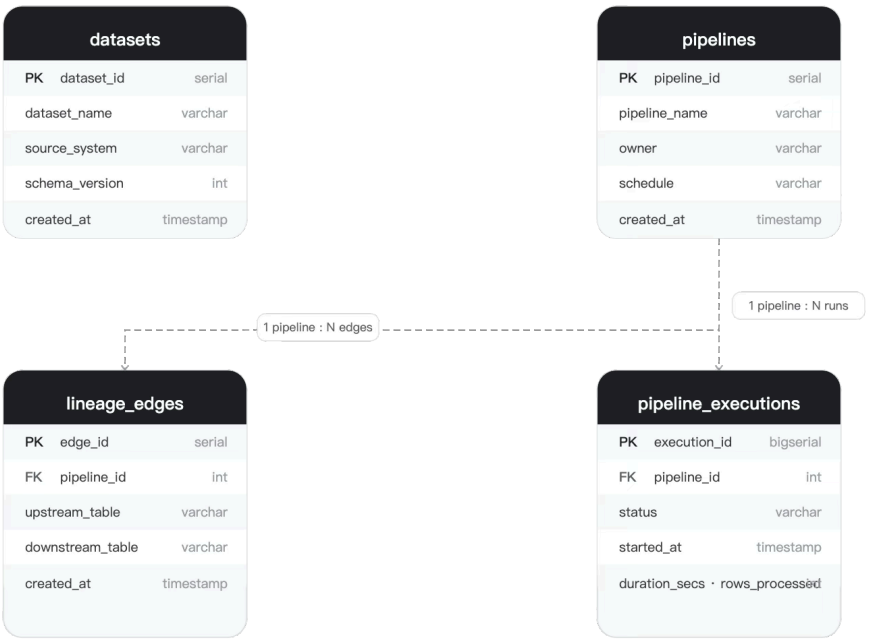
2. Database System Overview

2.1 Schema Overview

Four tables were created to support the Data Pipeline Monitoring & Lineage Tracking System and a Python script was written to generate and insert all synthetic data into PostgreSQL.

Table	Rows	Purpose
datasets	50	Stores metadata for source datasets
pipelines	100	Stores metadata for ETL pipelines
lineage_edges	200	Stores upstream/downstream table relationships
pipeline_executions	1,000,000	Stores time-series job execution logs with status & metrics

The ER diagram below illustrates the four-table structure and the two one-to-many relationships.



2.2 Table descriptions

2.2.1 pipelines — Central Entity

The `pipelines` table serves as the structural hub of the schema. Every ETL job in the system is registered here, along with its owner and cron-style schedule. Both `lineage_edges` and `pipeline_executions` reference `pipeline_id` as a foreign key, creating two distinct one-to-many relationships that together capture the full lifecycle of a pipeline: what it connects and how it runs.

```
CREATE TABLE pipelines (
    pipeline_id   SERIAL PRIMARY KEY,
    pipeline_name VARCHAR(100) NOT NULL,
    owner         VARCHAR(50),
    schedule      VARCHAR(50),
    created_at    TIMESTAMP DEFAULT NOW());
```

2.2.2 datasets — Decoupled from pipelines

The `datasets` table stores catalog-level metadata for the 50 source datasets in the system. Importantly, it does **not** carry a direct foreign key to `pipelines`. This decoupling is intentional: lineage relationships between tables are expressed through `lineage_edges` rather than through a rigid dataset-to-pipeline binding, which allows a single dataset to participate in multiple pipelines and enables graph traversal queries without schema changes.

```
CREATE TABLE datasets (
    dataset_id     SERIAL PRIMARY KEY,
    dataset_name   VARCHAR(100) NOT NULL,
    source_system  VARCHAR(50),
    schema_version INT DEFAULT 1,
    created_at     TIMESTAMP DEFAULT NOW());
```

2.2.3 lineage_edges — Graph-Oriented Design

The `lineage_edges` table stores directed edges in the data lineage graph. Each row records a `pipeline_id` alongside the names of an upstream and a downstream table, representing the data flow defined by that pipeline. This design enables recursive Common Table Expression (CTE) queries to traverse multi-hop upstream and downstream dependencies, which is a core feature of the lineage tracking system.

```
CREATE TABLE lineage_edges (
    edge_id          SERIAL PRIMARY KEY,
    pipeline_id      INT REFERENCES pipelines(pipeline_id),
    upstream_table   VARCHAR(100) NOT NULL,
    downstream_table VARCHAR(100) NOT NULL,
    created_at       TIMESTAMP DEFAULT NOW());
```

2.2.4 pipeline_executions — Time-Series at Scale

The `pipeline_executions` table is the highest-volume table in the schema, holding 1 million+ rows that simulate one year of hourly job runs. Each row logs the result of a single pipeline execution: its status, start and end timestamps, duration in seconds, and rows processed. Several design decisions reflect the demands of this scale:

- **BIGSERIAL for execution_id:** a standard SERIAL (INT) overflows at 2.1 billion rows. BIGSERIAL (BIGINT) supports up to 9.2×10^{18} rows, making it safe for production-scale ingestion.
- **Nullable ended_at and duration_secs:** rows with status 'running' have no end time yet, so these fields are intentionally nullable rather than defaulting to a sentinel value.

- **BRIN index candidate:** because execution records are written in approximately chronological order, the started_at column is a strong candidate for a BRIN (Block Range INdex) rather than a B-tree index — BRIN is far more storage-efficient for naturally ordered time-series data.

```
CREATE TABLE pipeline_executions (
    execution_id   BIGSERIAL PRIMARY KEY,
    pipeline_id    INT REFERENCES pipelines(pipeline_id),
    status         VARCHAR(20) CHECK (status IN ('success','failed','running')),
    started_at     TIMESTAMP NOT NULL,
    ended_at       TIMESTAMP,
    duration_secs  INT,
    rows_processed INT);
```

3. Internal Architecture

This project investigates three PostgreSQL internals: **Indexing, MVCC, and Recursive Query Execution**. Each focus area addresses a distinct challenge: fast lookups over a million-row time-series table, non-blocking reads during continuous writes, and efficient graph traversal over a pipeline dependency graph.

3.1 Heap and Page Structure

PostgreSQL uses a heap storage model where table data is stored in 8 KB pages on disk. Each row (tuple) is written to a heap page sequentially. This model has direct implications for index design: because rows are not physically sorted by any key, secondary indexes must store pointers (tuple IDs, or tids) back to the heap to retrieve the actual row data. After VACUUM updates the visibility map, PostgreSQL can use Index Only Scans fetch data entirely from the index without touching the heap which dramatically reduces I/O.

3.2 Indexing Strategies: B-tree vs. BRIN(Block Range Index)

B-tree indexes are the default and are suitable for high-cardinality columns with arbitrary access patterns. They organize key values in a balanced tree structure, providing O(log n) lookup for point queries and range scans regardless of data ordering. In this schema, B-tree indexes are appropriate for foreign key columns such as pipeline_id and lineage_edges, where the query planner needs to efficiently resolve joins.

BRIN (Block Range INdex) indexes are a fundamentally different structure designed for naturally ordered data. Rather than indexing individual row values, BRIN stores the minimum and maximum value of a column for each contiguous block range (128 pages by default). When a query filters by a range predicate, the planner uses the BRIN summary to skip entire block ranges that cannot contain matching rows. BRIN indexes are orders of magnitude smaller than B-tree indexes on the same column because they store one summary entry per block range rather than one entry per row. *The started_at column is a strong BRIN candidate because records are inserted in chronological order.*

3.3 Concurrency via MVCC

PostgreSQL uses Multi-Version Concurrency Control (MVCC) to handle concurrent access without locking. When a row is updated or deleted, PostgreSQL does not overwrite the existing data in-place. Instead, it writes a new version of the row called a tuple with a new transaction ID stamped in the `xmin` system column. The old tuple remains on disk with its original `xmax` set to the deleting transaction's ID. Readers see whichever version was committed as of their transaction's snapshot timestamp; they never block on writers, and writers never block readers.

3.3 Recursive CTE Execution for Lineage Traversal

The lineage query finding all upstream or downstream dependencies of a given table requires graph traversal across an arbitrary number of hops. PostgreSQL supports this natively through recursive CTEs, introduced with the WITH RECURSIVE syntax. A recursive CTE consists of two parts: an anchor member

(the base case, returning the seed row) and a recursive member (joined back to the CTE itself), separated by UNION ALL. The query planner materializes each iteration into a work table and repeats until no new rows are produced.

4. Application Design

4.1 Feature 1 — Lineage Graph Query

The application runs the recursive CTE query against `lineage_edges` and returns the full dependency chain with hop depth. Given a table name or pipeline ID, the application traces the full upstream and downstream dependency chain in a single command. Upstream tracing identifies root causes when a table contains bad data. Downstream tracing assesses the blast radius when a source table fails. The output also labels tables with schema_version, indicating frequent structural changes that may silently indicate unsolved root problems.

4.2 Feature 2 — Pipeline Monitoring

The application queries `pipeline_executions` with a given pipeline ID and time window to report on each pipeline's most recent run status, average duration over a configurable window, failure rate, and rows processed. This feature exercises both the B-tree index on `pipeline_id` (for per-pipeline lookups) and the BRIN index on `started_at` (for time-window filtering). MVCC ensures that live dashboard queries never stall on concurrent execution log inserts.

4.3 Feature 3 — Data Quality Tracking

The application tracks schema version changes (via `schema_version` in `datasets`) and monitors data freshness by checking the most recent successful execution timestamp for the fleet-wide health dashboard queries all 100 pipelines. Anomaly detection will flag pipelines that have not run within their expected schedule window. This is the starting point of the diagnostic workflow. A data engineer runs this each morning to identify which pipelines need attention before drilling into specifics with Features 1 and 2.

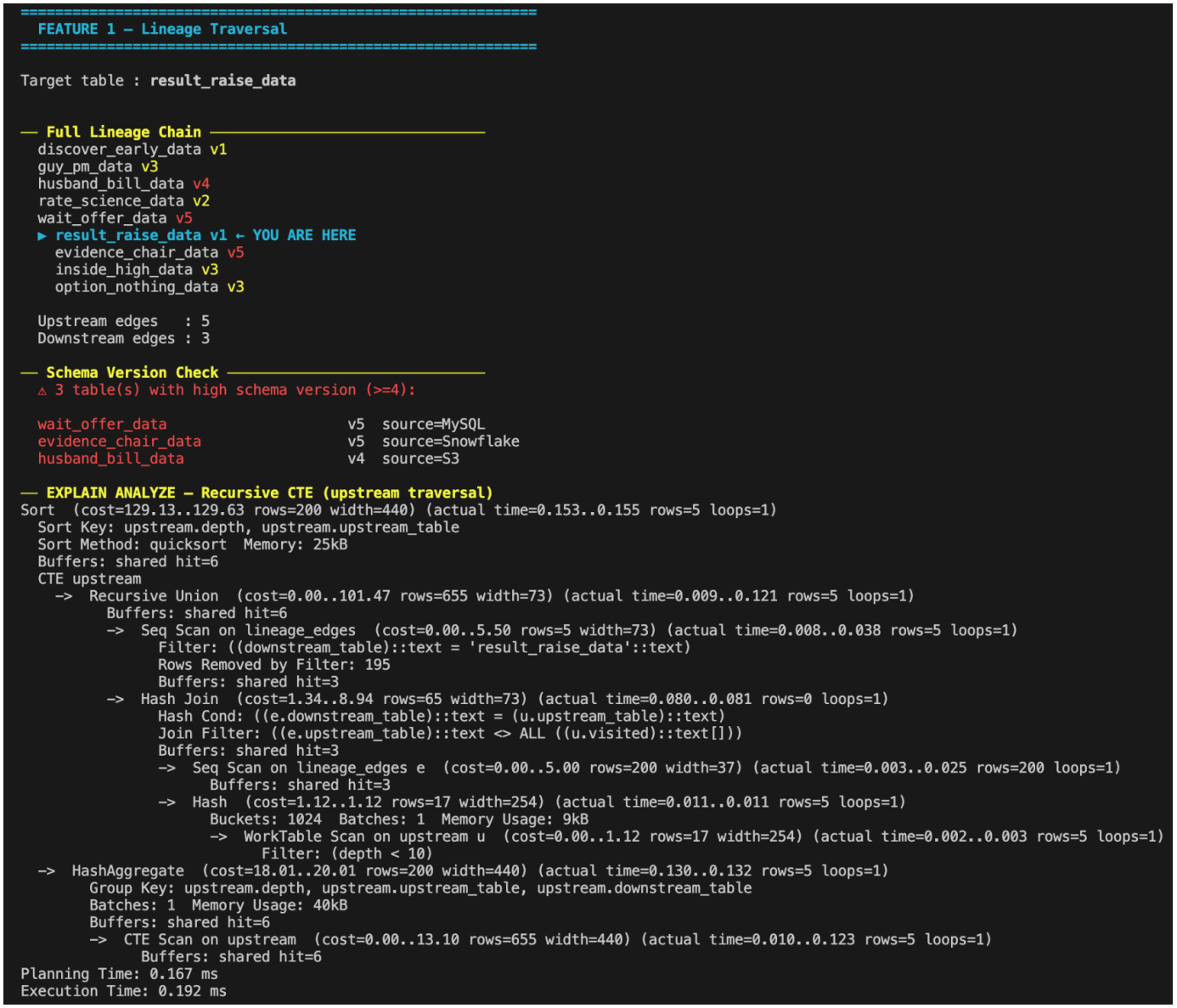
5. Mapping Internals to Application Behavior

Feature	DB Internal Mechanism	Application Behavior
Lineage Graph Query	`Recursive CTE work table`	Multi-hop lineage traversal returns complete dependency chain
Pipeline Monitoring	`B-tree index on pipeline_id`	Per-pipeline execution history lookups use index scan, not seq scan
Data Quality Tracking	`BRIN index on started_at`	Time-range queries (e.g. 'last 24h') skip most of the 1M-row heap
	`MVCC tuple versioning`	Dashboard reads never block during bulk execution log inserts

5.1 Recursive CTE — Lineage Traversal Detail

The EXPLAIN ANALYZE output for the lineage query shows Recursive Union with WorkTable Scan on each iteration. The anchor member executes once, seeding the first hop into the work table. The recursive member reads the work table, joins against lineage_edges to find the next hop, and appends results to an intermediate table. The work table is then replaced by the current iteration's output, and the process repeats. The final result is a CTE Scan over the accumulated intermediate table.

Example input: python main.py lineage --table result_raise_data



5.2 B-tree Index — Pipeline Lookup Detail

The EXPLAIN ANALYZE output for the pipeline inspect query shows Index Only Scan using idx_btree_pipeline_id with Heap Fetches: 0 after VACUUM. This means PostgreSQL retrieves approximately 10,000 execution records per pipeline entirely from the index without touching a single heap page. Execution time is consistently under 5 milliseconds on a one-million-row table. Without the index, the same query would require a sequential scan of all 8,000+ heap pages and cost approximately 250 milliseconds in average.

Example Input: **python main.py inspect --pipeline-id 78 --days 30**

```
============================================================
  FEATURE 2 — Pipeline Inspect
============================================================

Pipeline ID  : 78
Time window  : last 30 day(s)


— Pipeline Summary ——————————————————————
  Pipeline     : election_hope_pipeline
  Owner        : carol
  Schedule     : 0 */6 * * *
  Total runs   : 2,071
  Successes    : 1,669
  Failures     : 350  (16.9%)
  Avg duration : 1812 s
  Last run     : 2026-04-18 19:27:51.660105

— Last 10 Runs ——————————————————————————
  Status     Started At                  Duration(s)    Rows Processed
  ---------- --------------------------- ------------- ---------------
  success    2026-04-18 19:27:51.660105  1098          45,741
  success    2026-04-18 19:20:08.660105  2009          242,203
  failed     2026-04-18 19:17:20.660105  1794          N/A
  failed     2026-04-18 18:42:07.660105  2478          N/A
  success    2026-04-18 18:25:59.660105  976           95,213
  success    2026-04-18 18:23:18.660105  2921          298,970
  failed     2026-04-18 18:22:27.660105  2490          N/A
  failed     2026-04-18 18:01:16.660105  1697          N/A
  success    2026-04-18 17:57:47.660105  781           483,387
  success    2026-04-18 17:51:14.660105  1860          38,179

— EXPLAIN ANALYZE — B-tree Index on pipeline_id ————
Aggregate  (cost=237.77..237.78 rows=1 width=8) (actual time=3.324..3.324 rows=1 loops=1)
  Buffers: shared hit=11
  ->  Index Only Scan using idx_btree_pipeline_id on pipeline_executions  (cost=0.42..212.60 rows=10067 width=0) (actual time=2.169..2.820 rows=9962 loops=1)
        Index Cond: (pipeline_id = 78)
        Heap Fetches: 0
        Buffers: shared hit=11
Planning:
  Buffers: shared hit=3
Planning Time: 0.118 ms
Execution Time: 3.344 ms
```

5.3 BRIN Index — Time-Range Query Detail

The EXPLAIN ANALYZE output for the overview query with a 30-day window shows Bitmap Index Scan on idx_brin_started_at followed by Parallel Bitmap Heap Scan. The BRIN index reads only 2 shared buffers to identify the relevant block ranges, then the heap scan accesses approximately 1,890 blocks rather than the full 8,000+ pages. Rows Removed by Index Recheck are minimal, approximately 11,000 rows confirming that the BRIN summaries are tight and non-overlapping due to chronological insert ordering. Execution time went from 320 milliseconds to 105 milliseconds on average.

Example Input: **python main.py overview --days 30**

```
============================================================
  FEATURE 3 — Pipeline Health Overview
============================================================

Time window  : last 30 day(s)


— FAILED — 30 pipeline(s) ——————————————————
  ID   Pipeline                        Owner    Failures   Last Run
  ---- ------------------------------- -------- ---------- -------------------
  100  analysis_foot_pipeline          frank    358        2026-04-18 19:47:42.660105
  4    about_seven_pipeline            heidi    353        2026-04-18 19:46:57.660105
  78   election_hope_pipeline          carol    350        2026-04-18 19:27:51.660105
— STALE — 70 pipeline(s) overdue ———————————————
  ID   Pipeline                        Owner    Overdue        Schedule
  ---- ------------------------------- -------- -------------- ---------------
  98   also_they_pipeline              frank    430.8h overdue @weekly
  30   western_almost_pipeline         dave     430.7h overdue @hourly
  87   develop_series_pipeline         alice    430.7h overdue 0 2 * * *
— HEALTHY — 0 pipeline(s) ——————————————————
  All running on schedule with no failures.
— EXPLAIN ANALYZE — BRIN Index on started_at ————
Finalize Aggregate  (cost=11677.23..11677.24 rows=1 width=8) (actual time=12.420..13.397 rows=1 loops=1)
  Buffers: shared hit=2037
  ->  Gather  (cost=11677.02..11677.23 rows=2 width=8) (actual time=12.366..13.393 rows=3 loops=1)
        Workers Planned: 2
        Workers Launched: 2
        Buffers: shared hit=2037
        ->  Partial Aggregate  (cost=10677.02..10677.03 rows=1 width=8) (actual time=10.583..10.583 rows=1 loops=3)
              Buffers: shared hit=2037
              ->  Parallel Bitmap Heap Scan on pipeline_executions  (cost=63.44..10464.68 rows=84936 width=0) (actual time=0.181..7.703 rows=67044 loops=3)
                    Recheck Cond: (started_at >= '2026-04-06 18:30:10.888915'::timestamp without time zone)
                    Rows Removed by Index Recheck: 5652
                    Heap Blocks: lossy=859
                    Buffers: shared hit=2037
                    ->  Bitmap Index Scan on idx_brin_started_at  (cost=0.00..12.48 rows=205485 width=0) (actual time=0.035..0.035 rows=20350 loops=1)
                          Index Cond: (started_at >= '2026-04-06 18:30:10.888915'::timestamp without time zone)
                          Buffers: shared hit=2
Planning:
  Buffers: shared hit=1
Planning Time: 0.054 ms
Execution Time: 13.412 ms
```

5.4 MVCC — Concurrent Read/Write Detail

The Data Quality Tracking overview runs a fleet-wide aggregate query across pipeline_executions while

the same table is continuously receiving new execution records from active pipelines. MVCC makes this possible without any locking between the reader and the writer.

Every row in pipeline_executions carries two hidden system columns: xmin, the transaction ID that inserted the row, and xmax, the transaction ID that deleted it (zero if the row is still live). When the overview dashboard query begins, PostgreSQL takes a snapshot of the current transaction ID. The query sees only rows whose xmin is less than or equal to the snapshot. Any rows being written by concurrent transactions have a higher xmin and are simply invisible to the reader. No lock is acquired and no wait occurs.

The practical consequence is that the monitoring dashboard can refresh continuously without stalling on pipeline writes. In a production environment where hundreds of pipelines are completing and writing execution records every minute, this guarantee is essential. Without MVCC as would be the case with MySQL InnoDB row-level locks on hot rows, the dashboard reader could be blocked waiting for in-progress writes to release their locks, causing visible latency spikes during peak ingestion periods.

Example Input: **python main.py overview --days 30**

```
── MVCC — Concurrent Read/Write (xmin / xmax) ──────
  exec_id      pipeline_id   status     xmin       xmax
  ---------- ------------ --------- --------- ---------
  1000000      68           success    1086       0
  999999       77           success    1086       0
  999998       23           success    1086       0
  999997       65           success    1086       0
  999996       67           success    1086       0
```

6. Comparison with MySQL and MongoDB

6.1 Indexing

MySQL's InnoDB engine lacks a BRIN equivalent. All secondary indexes in InnoDB are B-trees, making time-range queries on large append-only tables comparatively less storage-efficient. MongoDB's index types include a similar concept (clustered collections) but without the same degree of planner integration for range-skipping on ordered heaps.

6.2 Concurrency — MVCC vs Locking Models

By contrast, MySQL with the MyISAM storage engine uses table-level locking: a write to any row in a table blocks all concurrent reads of that table. InnoDB uses row-level locking which is better, but still causes read-write contention on hot rows. MongoDB uses document-level locking, which avoids most contention on separate documents but introduces overhead for multi-document operations. PostgreSQL's MVCC avoids all of these scenarios: readers are completely isolated from writers at the tuple level. However, one cost of MVCC is table bloat which is addressed with autovacuum. Tuning autovacuum aggressiveness will be a consideration in the later phases.

6.3 Recursive Query Execution

MySQL 8.0 introduced recursive CTE support, but its integration with the query planner remains notably weaker than PostgreSQL's: recursive queries cannot fully leverage B-tree indexes during each iterative scan, meaning deeper graph traversals may still degrade into per-iteration full table scans as the lineage graph grows.

MongoDB has no native recursive query construct. Multi-hop graph traversal requires application-level code or the $graphLookup aggregation operator, which has depth and memory limits. PostgreSQL's recursive CTE integrates with the cost-based query planner: if a B-tree index exists each recursive iteration uses an index scan rather than a full table scan, making traversal efficient even as the lineage graph grows.

7. Limitations & Lessons Learned

7.1 Synthetic Data Constraints

The dataset is entirely synthetic, generated by a Python script rather than ingested from a real pipeline scheduler. In a production environment, pipeline_executions records would be written automatically by a scheduler such as Airflow after each task completion, lineage_edges would be populated from dbt manifest.json or OpenLineage events at deployment time, and pipelines metadata would be registered through a CI/CD process. The synthetic data was carefully calibrated, 30% of pipelines have above-threshold failure rates, records are inserted in chronological order to enable BRIN, and the DAG structure prevents cycles but it cannot fully replicate the irregularities of real production data.

7.2 Recursive CTE and Dense Graphs

The initial lineage graph was generated with random edges, producing a highly dense and cyclic graph. With UNION ALL and no cycle prevention, the recursive CTE produced over 1.3 million result rows and took nearly 2 seconds with disk spill due to memory exhaustion. Two lessons emerged: first, a visited array per row is essential for correct cycle prevention at the iteration level, not just output deduplication; second, the DAG structure of real lineage graphs is a meaningful design constraint that must be enforced at data generation time, not assumed.

8. Project Code & Documentation

All project source code, database schema setup scripts, synthetic data generation script, and this final report are available at the following Github link: